

Aufgabe 1:

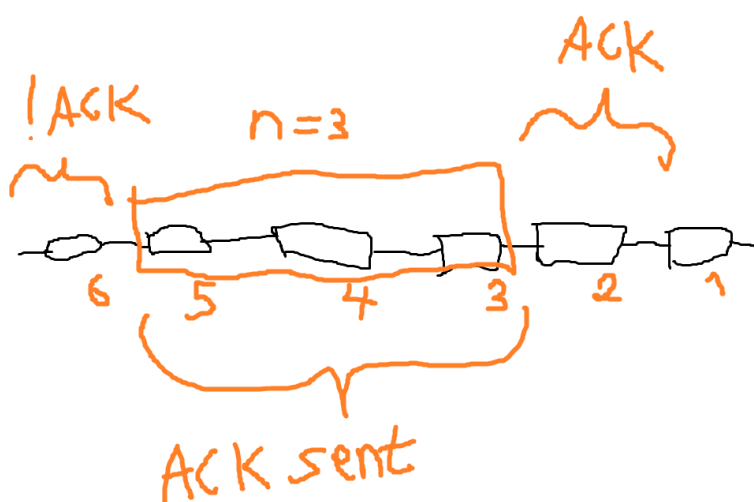
Sliding-Window: Das Sliding Window Protokoll wird verwendet, um den Quittungsbetrieb bei TCP-Paketen zu optimieren, also zur Flussssteuerung. Dabei wird ein Fenster mit einer Größe n definiert. Der Sender darf n Pakete senden ohne auf das ACK zu warten, um das Medium vollständig auszunutzen. Sobald das erste Paket ein ACK schickt, verschiebt sich das Fenster um eins und es kann ein weiteres Paket gesendet werden.

TCP Tahoe: Soll Überlastungen im Netzwerk verhindern. Slow start Eigenschaft, d.h. das Fenster ist zu Beginn 1 groß und wird dann exponentiell erhöht, um die Bandbreite zu ermitteln. Wenn ein Paket verloren geht, was durch dreimalig dupliertes ACK erkannt wird, wird eine Neusendung ausgelöst und die Fenstergröße auf 1 reduziert. Nutzt AIMD (Additive Increase / Multiplicative Decrease), d.h. die Fenstergröße wird linear erhöht, aber bei Kollisionserkennung exponentiell verringert.

TCP Reno: Weiterentwicklung von TCP Tahoe. Slow Start und Congestion Avoidance wie bei Tahoe. Die Fast Recovery erlaubt es uns den Slow Start Prozess zu verbessern, da wir nun die Fenstergröße halbieren.

TCP Vegas: basiert auf der Round Trip Time (RTT). Misst die tatsächliche und erwartete Zeit, die ein Paket brauchen sollte und bestimmt dann aus der Differenz, ob eine Überlastung vorliegt. Wenn das der Fall sein sollte, wird die Fenstergröße verringert. Wenn die RTT sinkt erhöhen wir die Round Trip time.

Die Grafik zeigt das Sliding Window Protokoll, wobei die Paket 1 und 2 bereits quittiert wurden, 3-5 ihr ACK gesendet haben, jedoch noch keins erhalten haben und 6 noch keines gesendet hat. Sobald sich das Fenster nach links verschiebt (wenn 3 ein ACK erhalten hat), schickt 6 ebenfalls sein Paket los.



Liste aller Protokolle:

Ebene 1 Physical Layer:

Ebene 2 Link Layer:

- FDMA – Frequency Division Multiple Access
- TDMA – Time Division Multiple Access
- CDMA – Code Division Multiple Access
- CSMA/CD – Carrier Sense Multiple Access / Collision Detect
 - Diese Protokolle werden alle dazu verwendet zu organisieren, wer wann senden darf und das Medium nutzt, weswegen ich diese auf der Link-Layer anordnen würde.
- Ethernet – Protokolle, die für die Adressierung auf dem physischen Medium mittels MAC-Adressen verwendet werden.

Ebene 3 Network Layer:

- IP-Protokoll / ICMP (Internet Control Message Protocol)
- ARP, RARP, da es aus IP-Adressen routbare MAC-Adressen macht und andersherum. Auf Ebene 2 werden die Daten an MAC-Adressen geschickt, weshalb diese unerlässlich sind.
- NAT (Network Address Translation), da es private IP-Adressen als routbare IP-Adressen zur Verfügung stellt.
- RIP (Routing Information Protocol)
- BGP (Border Gateway Protocol)

Ebene 4 Transport Layer:

- UDP
- TCP

Ebene 5 Session Layer:

Ebene 6 Presentation Layer:

Ebene 7 Application Layer:

- DNS (Domain Name System)
- DHCP (Dynamic Host Configuration Protocol)

Aufgabe 2 DHCP:

No.	Time	Delta	Source	Destination	Protocol	Length	Destination Port	Source Port	Info
1	0.000000	0.000000	0.0.0.0	255.255.255.255	DHCP	364	67	68	DHCP Request - Transaction ID 0x5167b38d
2	0.026788	0.026788	192.168.178.1	192.168.178.43	DHCP	590	68	67	DHCP ACK - Transaction ID 0x5167b38d
3	29.319410	29.292622	192.168.178.43	192.168.178.1	DHCP	342	67	68	DHCP Release - Transaction ID 0xc9803f5b
4	37.562879	8.243469	0.0.0.0	255.255.255.255	DHCP	344	67	68	DHCP Discover - Transaction ID 0x8c476a69
5	37.677332	0.114453	192.168.178.1	192.168.178.43	DHCP	590	68	67	DHCP Offer - Transaction ID 0x8c476a69
6	37.678298	0.000966	0.0.0.0	255.255.255.255	DHCP	370	67	68	DHCP Request - Transaction ID 0x8c476a69
7	37.687197	0.008899	192.168.178.1	192.168.178.43	DHCP	590	68	67	DHCP ACK - Transaction ID 0x8c476a69
8	48.234660	10.547463	192.168.178.43	192.168.178.1	DHCP	358	67	68	DHCP Request - Transaction ID 0x17f79b0d
9	48.243821	0.009161	192.168.178.1	192.168.178.43	DHCP	590	68	67	DHCP ACK - Transaction ID 0x17f79b0d

Damit man überhaupt DHCP Pakete empfangen kann, muss man den Netzwerkadapter ein und ausschalten und somit eine Verbindung zum Netzwerk aufbauen.

```
Destination Address: 255.255.255.255
▼ User Datagram Protocol, Src Port: 68, Dst Port: 67
  Source Port: 68
  Destination Port: 67
  Length: 330
  Checksum: 0xae25 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 0]
  ▶ [Timestamps]
  UDP payload (322 bytes)
▼ Dynamic Host Configuration Protocol (Request)
  Message type: Boot Request (1)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x5167b38d
  Seconds elapsed: 0
  ▶ Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 0.0.0.0
  Next server IP address: 0.0.0.0
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_01:df:3d (f8:e4:e3:01:df:3d)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
```

Dieses Paket ist ein DHCP Boot-request, d.h. die initiale Anfrage, dass man eine der Offers akzeptiert und sich mit der IP Adresse registriert. Man sieht zusätzlich, dass DHCP auf UDP basiert.

Hardwaretype ist Ethernet, die Länge der Hardwareadresse (MAC-Adresse) ist 6 Bytes.

Die Client IP Adresse ist noch 0.0.0.0, da noch keine IP Adresse zugewiesen wurde.

Ebenso sieht man die Mac-Adresse des Clients.

[illegible]

Hier haben wir dann ein Paket der Art ACK. Hier wird nun eine Ip Adresse zugewiesen 192.168.178.43, sowie alle weiteren Parameter die für das IP Lease notwendig sind.

```
[Stream index: 0]
  ▸ [Timestamps]
    UDP payload (302 bytes)
  ▾ Dynamic Host Configuration Protocol (Discover)
    Message type: Boot Request (1)
    Hardware type: Ethernet (0x01)
    Hardware address length: 6
    Hops: 0
    Transaction ID: 0x8c476a69
    Seconds elapsed: 0
    ▸ Bootp flags: 0x0000 (Unicast)
      Client IP address: 0.0.0.0
      Your (client) IP address: 0.0.0.0
      Next server IP address: 0.0.0.0
      Relay agent IP address: 0.0.0.0
      Client MAC address: Intel_01:df:3d (f8:e4:e3:01:df:3d)
      Client hardware address padding: 00000000000000000000
      Server host name not given
      Boot file name not given
      Magic cookie: DHCP
    ▸ Option: (53) DHCP Message Type (Discover)
    ▸ Option: (61) Client identifier
    ▸ Option: (50) Requested IP Address (192.168.178.43)
    ▸ Option: (12) Host Name
    ▸ Option: (60) Vendor class identifier
    ▸ Option: (55) Parameter Request List
    ▸ Option: (255) End
0000 ff ff ff ff ff ff f8 e4 e3 01 df 3d 08 00 45 00
```

Hier haben wir eine DHCP Discovery Nachricht, die vom Client gesendet wird. Eine Discovery Nachricht ist immer ein Broadcast, weil der Client anfragt, ob ein Server existiert, der ihm eine IP Adresse zuweisen kann.

```
[Coloring Rule String: udp]
Ethernet II, Src: Intel_01:df:3d (f8:e4:e3:01:df:3d), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
  Destination: Broadcast (ff:ff:ff:ff:ff:ff)
  Source: Intel_01:df:3d (f8:e4:e3:01:df:3d)
  Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 0.0.0.0, Dst: 255.255.255.255
  0100 .... = Version: 4
```

Zuletzt gibt es noch DHCP Offer Pakete, welche vom DHCP-Server an den Client geschickt werden (hier sichtbar daran, dass die Destination die MAC-Adresse meines Laptops ist).

Dieses Offer bietet dem Client eine IP-Adresse an mit Konfigurationsparametern, wie z.B. auch der Renewal Time.

```

Magic cookie: DHCP
Option: (53) DHCP Message Type (Offer)
  Length: 1
  DHCP: Offer (2)
Option: (54) DHCP Server Identifier (192.168.178.1)
  Length: 4
  DHCP Server Identifier: 192.168.178.1
Option: (51) IP Address Lease Time
  Length: 4
  IP Address Lease Time: 10 days (864000)
Option: (58) Renewal Time Value
  Length: 4
  Renewal Time Value: 5 days (432000)
Option: (59) Rebinding Time Value
  Length: 4
  Rebinding Time Value: 8 days, 18 hours (756000)
Option: (1) Subnet Mask (255.255.255.0)

[Coloring Rule String: udp]
Ethernet II, Src: AVMAudiovisu_a8:9c:4c (98:9b:cb:a8:9c:4c), Dst: Intel_01:df:3d (f8:e4:e3:01:df:3d)
  Destination: Intel_01:df:3d (f8:e4:e3:01:df:3d)
  Source: AVMAudiovisu_a8:9c:4c (98:9b:cb:a8:9c:4c)
  Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 192.168.178.1, Dst: 192.168.178.43
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 576
  Identification: 0x5ba5 (23461)
  000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: UDP (17)
  Header Checksum: 0x378a [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.178.1
  Destination Address: 192.168.178.43
User Datagram Protocol, Src Port: 67, Dst Port: 68
  Source Port: 67
  Destination Port: 68
  Length: 556
  Checksum: 0x1e91 [unverified]
  [Checksum Status: Unverified]
  [Stream index: 1]
  [Timestamps]
  UDP payload (548 bytes)
Dynamic Host Configuration Protocol (Offer)
  Message type: Boot Reply (2)
  Hardware type: Ethernet (0x01)
  Hardware address length: 6
  Hops: 0
  Transaction ID: 0x8c476a69
  Seconds elapsed: 0
  Bootp flags: 0x0000 (Unicast)
  Client IP address: 0.0.0.0
  Your (client) IP address: 192.168.178.43
  Next server IP address: 192.168.178.1
  Relay agent IP address: 0.0.0.0
  Client MAC address: Intel_01:df:3d (f8:e4:e3:01:df:3d)
  Client hardware address padding: 00000000000000000000
  Server host name not given
  Boot file name not given
  Magic cookie: DHCP
  Option: (53) DHCP Message Type (Offer)
  Option: (54) DHCP Server Identifier (192.168.178.1)
  Option: (51) IP Address Lease Time
  Option: (58) Renewal Time Value
  Option: (59) Rebinding Time Value
  Option: (1) Subnet Mask (255.255.255.0)

```

Insgesamt gibt es 4 verschiedene Arten von DHCP-Paketen.

1. DHCP Discovery:
2. DHCP Offer
3. DHCP Request
4. DHCP ACK

Aufgabe 3:

a) nmap -sn 192.168.0.0/24

```
Starting Nmap 7.95 ( https://nmap.org ) at 2024-07-10 14:14 Mitteleuropäische Sommerzeit
Stats: 0:01:05 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 31.74% done; ETC: 14:17 (0:02:20 remaining)
Stats: 0:01:07 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 32.71% done; ETC: 14:17 (0:02:18 remaining)
Stats: 0:01:08 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 33.11% done; ETC: 14:17 (0:02:17 remaining)
Stats: 0:01:08 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 33.20% done; ETC: 14:17 (0:02:17 remaining)
Stats: 0:03:10 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 92.28% done; ETC: 14:17 (0:00:16 remaining)
Stats: 0:03:12 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 93.26% done; ETC: 14:17 (0:00:14 remaining)
Stats: 0:03:13 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 93.75% done; ETC: 14:17 (0:00:13 remaining)
Stats: 0:03:14 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 93.99% done; ETC: 14:17 (0:00:12 remaining)
Stats: 0:03:14 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 94.19% done; ETC: 14:17 (0:00:12 remaining)
Stats: 0:03:14 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 94.24% done; ETC: 14:17 (0:00:12 remaining)
Stats: 0:03:15 elapsed; 0 hosts completed (0 up), 256 undergoing Ping Scan
Ping Scan Timing: About 94.34% done; ETC: 14:17 (0:00:12 remaining)
Nmap done: 256 IP addresses (0 hosts up) scanned in 207.52 seconds
```

b) nmap -O scanme.nmap.org

```
C:\Users\leon>nmap -O scanme.nmap.org
Starting Nmap 7.95 ( https://nmap.org ) at 2024-07-10 14:21 Mitteleuropäische Sommerzeit
Stats: 0:00:02 elapsed; 0 hosts completed (0 up), 1 undergoing Ping Scan
Parallel DNS resolution of: 1 host. Timing: About 0.00% done
Stats: 0:00:07 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 75.80% done; ETC: 14:21 (0:00:01 remaining)
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.16s latency).
Not shown: 993 closed tcp ports (reset)
Other addresses for scanme.nmap.org (not scanned): 2600:3c01:f03c:91ff:fe18:bb2f
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
135/tcp    filtered msrpc
139/tcp    filtered netbios-ssn
445/tcp    filtered microsoft-ds
9929/tcp   open  msrpc-echo
31337/tcp  open  Elite
Aggressive OS guesses: Linux 4.15 - 5.10 (90%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (90%), Linux 3.2 - 4.14 (93%), Linux 2.6.32 - 3.10 (92%), Linux 5.0 - 5.14 (92%), Linux 4.15 (91%), Linux 5.10 (91%), Li
ux 3.10 (91%), Linux 4.19 (91%), Linux 4.12 (91%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 19 hops
OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.90 seconds
```

Es wird vermutlich Linux verwendet.

c) Whois nmap.org

```
WHOIS Server: whois.dynadot.com
Registrar URL: http://www.dynadot.com
Updated Date: 2023-08-31T05:05:15Z
Creation Date: 1999-01-18T05:00:00Z
Registry Expiry Date: 2029-01-18T05:00:00Z
Registrar: Dynadot Inc
Registrar IANA ID: 472
Registrar Abuse Contact Email: abuse@dynadot.com
Registrar Abuse Contact Phone: +1.6502620100
Domain Status: clientTransferProhibited https://icann.org/epp#clientT
Registry Registrant ID: REDACTED FOR PRIVACY
Registrant Name: REDACTED FOR PRIVACY
Registrant Organization: Super Privacy Service LTD c/o Dynadot
Registrant Street: REDACTED FOR PRIVACY
Registrant City: REDACTED FOR PRIVACY
Registrant State/Province: California
Registrant Postal Code: REDACTED FOR PRIVACY
Registrant Country: US
Registrant Phone: REDACTED FOR PRIVACY
Registrant Phone Ext: REDACTED FOR PRIVACY
Registrant Fax: REDACTED FOR PRIVACY
Registrant Fax Ext: REDACTED FOR PRIVACY
```

d) `nmap -p 1-65535 -T4 -A -v 192.168.0.0/24`

Wobei `-p 1-65535` die TCP Ports 1-65535 scannt.

`-T4` erhöht die Geschwindigkeit des Scans.

`-A` Aktiviert erweiterte Scan-Techniken wie Betriebssystemerkennung, Versionskennung und Traceroute.

`-v` Erhöht die Ausgabe Details.

104	7.701878	0.001083	192.168.178.43	192.168.0.38	ICMP	42				tcno (ping) request id=0x0b8a, seq=0/0, ttl=48 (no response found!)
105	7.824039	0.122161	54.220.108.215	192.168.178.43	TCP	54	52822	443	443 → 52822 [ACK] Seq=1 Ack=1 Win=478 Len=0	
106	7.824085	0.000046	192.168.178.43	54.220.108.215	TCP	54	443	52822	[TCP ACKed unseen segment] 52822 → 443 [ACK] Seq=1 Ack=2 Win=514 Len=0	
107	7.990364	0.166279	2003:c9:df11:c9...	2606:4700:4400:...	UDP	103	443	53609	53609 → 443 Len=41	
108	8.044903	0.004539	2606:4700:4400:...	2003:c9:df11:c9...	UDP	85	53609	443	443 → 53609 Len=23	
109	8.191135	0.146232	192.168.178.43	192.168.0.61	ICMP	42				Echo (ping) request id=0xebe7, seq=0/0, ttl=40 (no response found!)
110	8.234805	0.043670	192.168.178.43	192.168.0.62	ICMP	42				Echo (ping) request id=0x4d51, seq=0/0, ttl=49 (no response found!)
111	8.236482	0.001677	192.168.178.43	192.168.0.63	ICMP	42				Echo (ping) request id=0x4b71, seq=0/0, ttl=54 (no response found!)
112	8.238370	0.001888	192.168.178.43	192.168.0.64	ICMP	42				Echo (ping) request id=0x01a4, seq=0/0, ttl=37 (no response found!)
113	8.252490	0.014120	192.168.178.43	192.168.0.67	ICMP	42				Echo (ping) request id=0x5ccd, seq=0/0, ttl=55 (no response found!)
114	8.254161	0.001671	192.168.178.43	192.168.0.68	ICMP	42				Echo (ping) request id=0x9a37, seq=0/0, ttl=46 (no response found!)
115	8.255783	0.001622	192.168.178.43	192.168.0.69	ICMP	42				Echo (ping) request id=0xcbb6, seq=0/0, ttl=40 (no response found!)
116	8.257544	0.001761	192.168.178.43	192.168.0.70	ICMP	42				Echo (ping) request id=0xb670, seq=0/0, ttl=48 (no response found!)
117	8.258352	0.000000	192.168.178.43	192.168.0.71	ICMP	42				Echo (ping) request id=0xe555, seq=0/0, ttl=39 (no response found!)
118	8.259146	0.000794	192.168.178.43	192.168.0.72	ICMP	42				Echo (ping) request id=0xcd1b, seq=0/0, ttl=48 (no response found!)
119	8.440815	0.380869	2003:c9:df11:c9...	2600:1901:1:ca7...	TLSv...	117	443	50746	Application Data	
120	8.662405	0.022390	2600:1901:1:ca7...	2003:c9:df11:c9...	TCP	74	50746	443	443 → 50746 [ACK] Seq=1 Ack=44 Win=290 Len=0	
121	8.674253	0.011848	2600:1901:1:ca7...	2003:c9:df11:c9...	TLSv...	114	50746	443	Application Data	
122	8.702174	0.027921	192.168.178.43	192.168.0.25	ICMP	42				Echo (ping) request id=0xdfb2, seq=0/0, ttl=48 (no response found!)
123	8.716825	0.014651	2003:c9:df11:c9...	2600:1901:1:ca7...	TCP	74	443	50746	50746 → 443 [ACK] Seq=44 Ack=41 Win=1027 Len=0	
124	8.748201	0.031376	192.168.178.43	192.168.0.26	ICMP	42				Echo (ping) request id=0x426f, seq=0/0, ttl=45 (no response found!)
125	8.857617	0.109416	192.168.178.43	192.168.0.27	ICMP	42				Echo (ping) request id=0x6aa0, seq=0/0, ttl=51 (no response found!)
126	8.858248	0.000631	192.168.178.43	192.168.0.28	ICMP	42				Echo (ping) request id=0x5c08, seq=0/0, ttl=55 (no response found!)
127	8.858808	0.000560	192.168.178.43	192.168.0.29	ICMP	42				Echo (ping) request id=0xbe65, seq=0/0, ttl=55 (no response found!)
128	8.859369	0.000561	192.168.178.43	192.168.0.30	ICMP	42				Echo (ping) request id=0x968c, seq=0/0, ttl=50 (no response found!)
129	8.859931	0.000562	192.168.178.43	192.168.0.31	ICMP	42				Echo (ping) request id=0x303d, seq=0/0, ttl=50 (no response found!)

Hierbei sieht man in Wireshark sehr viele Pings an die Ports.

e) `nmap -sS 192.168.0.1`

SYN-Scan sendet ein TCP-SYN Paket zum Empfänger, wobei dieser ein SYN-ACK-Paket zurückschickt, wenn der Port offen ist.

f) `nmap -p 80,443,22,21,25 192.168.0.0/24`

Häufig offene Ports sind 80 (HTTP), 443 (HTTPS), 22 (SSH), 21 (FTP), 25 (SMTP).

Aufgabe 4:

Dynamische oder Adaptive Routing kann im Vergleich zum statischen Routingverfahren eigenständig neue Routen finden, bspw. Durch geschädigte Leitungen usw.

Distanzverkoralgorithmien sind dynmische Routingprotokolle, die dem Nachbarn mitteilen, wie sie die „Welt“, also das Netzwerk sehen. Diese basieren auf dem Bellman-Ford-Algorithmus. Distanzbasierte sind zum Beispiel RIP.

Der Unterschied zum Link-State Verfahren besteht darin, dass beim Distanzvektorverfahren jeder Router nur die Kosten zu seinem direkten Nachbarn kennt und beim Link-State Verfahren kennt jeder Router die gesamte Netzwerktopologie und Kosten zu allen Routern, was dann präzisere Berechnungen ermöglicht und Dijkstras Algorithmus nutzt.

In der vorliegenden Routingtabelle läuft es wie folgt ab. Router A kennt die Kosten zu B und C initial. In der 1. Aktualisierung erhält er Informationen von B darüber, was die Kosten von B zu C sind und addiert diese mit den Kosten zu B (3) zu 5 auf. Das gleiche passiert für den Weg zu B über C. Der Weg zu D über C wird ebenfalls von C mitgeteilt. Im nächsten Schritt erhält A von B die Informationen, dass der Weg zu D über B 7 kostet, inklusive der Kosten zu B (3) = 10.

Ein potenzieller Algorithmus wäre:

1. Jeder Router initialisiert seine Routingtabelle zu seinen Nachbarn.
2. While (alle Felder in der Routing Tabelle noch nicht ausgefüllt sind tausche dich mit deinen Nachbarn aus): Aktualisiere die Routingtabelle und vergleiche die empfangenen Kosten mit den aktuellen und falls die empfangenen kleiner sind, aktualisiere die Tabelle.

b) der Kostengünstigste Pfad von z nach x bleibt bestehen.

c) C wird das Ausfallen des Routers direkt mitbekommen, wenn die Tabellen aktualisiert werden. Die Router B und A erst danach, da diese nicht die gesamte Netzwerktopologie kennen und sich nur auf Router C beziehen können.